MarketCatalyst — AI Feature: Framework, Plan & Tools

Purpose: how we add AI-generated insight across the app — the architecture, the phased plan, the tools, the cost model, and the guardrails — before implementation proceeds. Date: 2026-08-19.

Goal in one line: AI read-outs that explain *why* a stock/market is moving by **synthesising the technical indicators with the latest news**, delivered on-demand, cached, and **appended to existing widgets** (never replacing them).

---

1. Design principles (the guardrails)

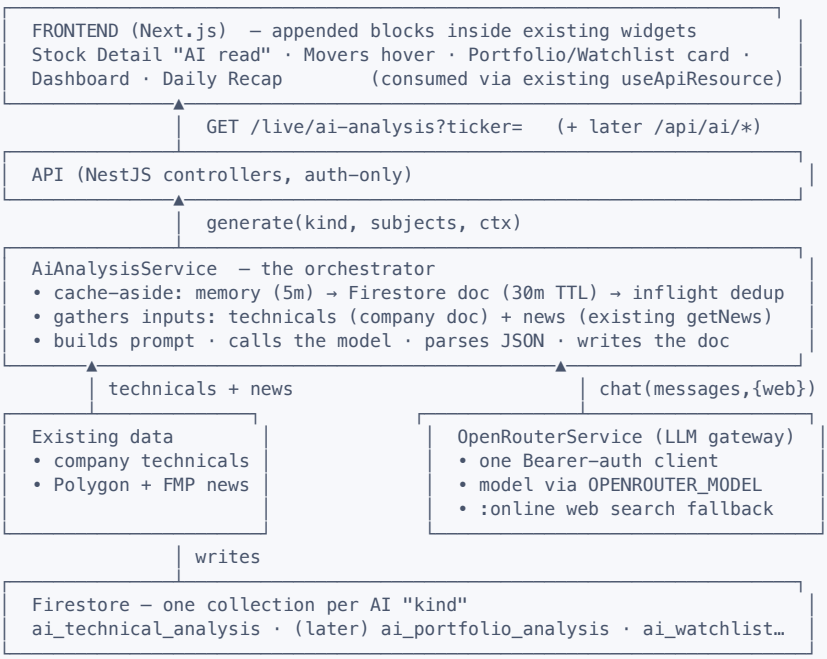
These hold for every phase.

Principle	What it means
Additive only	AI content is **appended inside the existing widget**. No existing UI is removed, no layout replaced.
No data deletion	The AI layer only **reads** data the app already has and **writes** new AI docs to new collections. Nothing existing is deleted or overwritten.
Technicals + news together	Every read combines the **technical-indicator snapshot** with the **news**, and flags where they agree or diverge — not a news summary alone.
On-demand + cached	Generated only when a user opens the view; cached 30 min. No cron jobs. Controls cost and latency.
Informational only	No buy/sell/hold advice, no price targets. Every read carries a disclaimer.
Untrusted output	Model output is rendered as **plain text** (never as HTML), so a model can't inject markup.
Graceful degradation	No key / model failure ⇒ "AI unavailable", never a crash.
Open to all (for now)	No premium gate today; auth-only. Premium-gating is a one-line future toggle.
Privacy-scoped	Per-user reads (portfolio/watchlist) are scoped to the verified `uid`, never a client-supplied id.

---

2. Framework (architecture)

Five layers, each reusing patterns that already exist in the codebase:



Why it's low-risk: every layer clones an existing pattern — the cache-aside is copied from `OnDemandService.getTranscript()`, the vendor is copied from the FMP service, the endpoint from the existing on-demand routes, the frontend from the existing `useApiResource` hook. We add code; we don't rewire anything.

Request flow (one AI read)

1. User opens a stock → frontend calls `GET /live/ai-analysis?ticker=AAPL` (auth token attached automatically).
2. `AiAnalysisService`: memory cache hit? Firestore doc < 30 min old? → **serve cached**.
3. Else: gather **technicals** (from the company doc) + **news** (existing `getNews`). If **no first-party news** → use the model's `:online` web search.
4. Build the prompt → **OpenRouter** → parse JSON → **write the doc** (new `createdAt`) → return.
5. Frontend renders the appended "AI read" block (loading → populated).

---

3. Tools & stack

Concern	Tool / choice	Notes
LLM gateway	**OpenRouter**	One account/key reaches many models. Bearer auth.
Model	`OPENROUTER_MODEL` env var, default **free** (`deepseek/deepseek-chat-v3:free`)	Swap to a paid model per-environment with zero code change.
News fallback	OpenRouter `:online` (web-search plugin)	When Polygon+FMP have no news, the model fetches recent web results itself.
Backend	**NestJS** (existing)	New vendor module + one service + one route.
Cache	**Firestore** + in-memory map	30-min TTL doc per subject; 5-min hot memory; inflight promise dedup.
Data inputs	Existing **Polygon + FMP** news, **company technicals**	No new data vendors.

Concern	Tool / choice	Notes
Frontend	**Next.js** (existing hooks/components)	`useApiResource` + appended widget blocks.
Secrets	**GCP Secret Manager** → `apphosting.yaml` / Cloud Run	`OPENROUTER_API_KEY`.
Deploy	`gcloud run deploy` (backend) + `firebase deploy --only hosting` (UI)	Existing pipeline.
(Later) Video/audio	HeyGen / ElevenLabs	Separate initiative; not in the core phases.

---

4. Plan for achievement (phased)

Each phase reuses the **same** `OpenRouterService` + `AiAnalysisService.generate(kind,…)` + cache pattern. Only a new prompt + context builder + a frontend block are added per phase.

Phase	Scope	Status
0 · Foundation	OpenRouter vendor, AiAnalysisService, cache pattern, endpoint, config	**Built (code complete, builds clean)**
1 · Stock Detail	AI read (technicals + news) **appended** to the AI Technical Analysis card: volatility, momentum (up/down/bear), news summary, support/patterns/consolidation/accumulation	**Built — needs the API key + deploy**
2 · Movers	"Why it's moving" one-liner in the movers hover popup + drawer (top-5 news + move context)	Planned
3 · Portfolio & Watchlist	On-demand AI summary + risk indicators (sector concentration, cost-basis variance) in the **existing AiSummaryCard**; per-user, scoped to uid	Planned
4 · Dashboard	"What's happening now" (macro + events + headlines); most-searched / mover hover micro-summaries	Planned
5 · Daily Recap	"How indices moved & why" + curated news outcomes	Planned
6 · Copilot	Wire the existing (stubbed) Copilot panel to an AI endpoint	Planned
7 · Social video/audio	EOD summary → short-form video (HeyGen/ElevenLabs) for social	Separate later initiative

---

5. Cost model & controls

- **Free model = $0** (subject to OpenRouter free-tier daily limits). **Paid low-cost** model is cheap and higher-limit if we need reliability.
- `:online` **web search** is a small paid add-on **per fallback call only** (used only when a ticker has no first-party news).
- **Caching is the main cost control:** one generation per subject per **30 minutes**, regardless of how many users view it; a 5-min memory layer + inflight dedup collapse bursts.
- **Auth gate** prevents anonymous users triggering paid calls.
- Net effect: cost scales with *distinct subjects viewed per 30-min window*, not with traffic.

6. What we need to start

One thing: an **OpenRouter API key**, stored as a Secret Manager secret `OPENROUTER_API_KEY`. Everything else (model, endpoints, UI, cache) is already coded and configurable. Until the key is present, the feature returns "AI unavailable" and nothing breaks.

Go-live steps (Phase 1): create the secret → add key+model to the live Cloud Run service → `gcloud run deploy market-catalyst-live --source .` → `firebase deploy --only hosting` → coordinate timing with the in-progress FMP-sector refactor.

Appendix — files (Phase 1)

- **New:** `src/vendors/openrouter/openrouter.{service,module}.ts`, `src/live/ai-analysis.service.ts`, `app/iq/types/ai.ts`
- **Registration/additions only:** `src/live/live.module.ts`, `src/live/ondemand.controller.ts` (one route), `apphosting.yaml` (env), `app/iq/screens/stock.tsx` (one appended block + one hook)
- **New Firestore collection:** `ai_technical_analysis` (per-ticker; `createdAt` TTL)